

Hashing Large Objects

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Hashing: Much More than Just a Data Structure!

- The general idea of mapping a large, complicated object down to a simpler object appears in many areas of computing...
- Example: hashing a file down to a small integer “fingerprint” lets us:
 - Compare (approximately) two files extremely quickly.
 - Test if two files at different locations (e.g., original, cloud backup) are identical with a minimum amount of communication.
 - Detect tampering in critical system files.
 - Ensure integrity of the file when transferred over a noisy channel, by appending the fingerprint as a checksum.
 - Identify duplicate files, possibly to save on storage space.

Hashing Arrays / Large Objects

- Any complicated object can be “serialized” and represented by an array of small integers.
E.g., “Cpsc” $\rightarrow$ ‘C’, ‘p’, ‘s’, ‘c’ $\rightarrow$ 67, 112, 115, 99.
- So how do we hash an array $A[0 \dots N - 1]$ to get an output value in the range $0 \dots M - 1$?

Hashing Arrays

- Any complicated object can be “serialized” and represented by an array of small integers.
E.g., “Cpssc” $\rightarrow$ ‘C’, ‘p’, ‘s’, ‘c’ $\rightarrow$ 67, 112, 115, 99.
- So how do we hash an array $A[0 \dots N - 1]$ to get an output value in the range $0 \dots M - 1$?
- How about something like this:
$$h(A[]) = (A[0] + A[1] + \dots + A[N - 1]) \bmod M$$

Polynomial Hash Functions

- Think of $A[]$ as the coefficients of a polynomial:

$$p_A(x) = A[0] + A[1]x + A[2]x^2 + \dots + A[N-1]x^{N-1}$$

- To hash $A[]$, evaluate $p(x) \bmod M$ at a randomly-chosen point x .
How do we do this quickly?
 - Horner's rule: $p_A(x) = A[0] + x (A[1] + x (A[2] + x (A[3] + \dots)))$
 - For $i = N-1$ to 0 : $\text{hash} = (\text{hash} * x + A[i]) \bmod M$
- Taking “mod M ” as we go equivalent to taking “mod M ” once at end.
 - Do this to keep numbers manageable in size during our calculations.

Polynomial Hash Functions

- Think of $A[]$ as the coefficients of a polynomial:

$$p_A(x) = A[0] + A[1]x + A[2]x^2 + \dots + A[N-1]x^{N-1}$$

- To hash $A[]$, evaluate $p(x) \bmod M$ at a randomly-chosen point x .
- If M is prime(?!), then the probability two different arrays A and B collide at most $(N-1) / M$:
 - If A and B collide, then x is a root of $p_A(x) - p_B(x) \pmod{M}$.
$$p_A(x) - p_B(x) = (A[0] - B[0]) + (A[1] - B[1])x + \dots$$
 - A polynomial of degree $N - 1$ can have at most $N - 1$ roots (true in any “algebraic field”, such as arithmetic over complex numbers or over integers modulo a prime). So only $N - 1$ “bad” values of x in $\{0, 1, 2, \dots, M - 1\}$.

Polynomial Hash Functions

- Think of $A[]$ as the coefficients of a polynomial:

$$p_A(x) = A[0] + A[1]x + A[2]x^2 + \dots + A[N-1]x^{N-1}$$

- To hash $A[]$, evaluate $p(x) \bmod M$ at a randomly-chosen point x .
- If M is prime(?!), then the probability two different arrays A and B collide at most $(N-1) / M$
- Easy to remember in terms of “back of envelope” math: probability of collision $\leq N / M = \text{array size} / \text{output range size}$

Application: String Matching

- **Input:**

- Text T of length N (usually quite long).
- Pattern P (usually much shorter).

- **Goal:** Find occurrences of the pattern in the text:

6 14 16 24
↓ ↓ ↓ ↓
 $T = \text{ACGTGATACTGCTATATACATCGATATGCTCA}$
 $P = \text{ATA}$

- String matching is a “classical” algorithm problem with many applications in practice. For example,
 - Searching the human genome ($N \approx 3$ billion).
 - Searching large documents (e.g., word processing) or entire document collections (e.g., the web).

String Matching with Hashing

- Karp-Rabin string matching: compare hash of pattern with running hash of a sliding window through the text.

T = ACGTGATACTGCTATATACATCGATATGCTCA
P = ATA

- A polynomial hash function is ideal here, since we can update the hash of the sliding window at each step in just $O(1)$ time!

$$p(x) = (A[0] x^{N-1} + A[1] x^{N-2} + \dots + A[N-2] x + A[N-1]) \bmod M$$

$$p'(x) = (A[1] x^{N-1} + A[2] x^{N-2} + \dots + A[N-1] x + A[N]) \bmod M$$

$$= [x(p(x) - A[0] x^{N-1}) + A[N]] \bmod M$$

Aside: Multiplicative Inverses Working Modulo a Prime

- The previous slide used a polynomial hash function written “backwards”:

$$(A[0] x^{N-1} + A[1] x^{N-2} + \dots + A[N-2] x + A[N-1]) \bmod M$$

- This allowed each step to multiply by x .
- Otherwise, we would have had to “divide” by x at each step.
- Surprisingly, though, this isn’t hard to do!
 - As long as M is prime, then every integer x in $\{1, 2, \dots, M - 1\}$ has a unique multiplicative inverse x^{-1} such that $xx^{-1} \bmod M = 1$.
 - E.g., you can solve equations like $3x \equiv 4 \pmod{7}$ by multiplying both sides by $3^{-1} \equiv 5 \pmod{7}$, giving $x \equiv 6 \pmod{7}$.
 - Euclid’s greatest common divisor algorithm can find x^{-1} quickly.

Substring Hashing via Difference of Two “Prefix Hashes”

- Suppose we hash an array $A[]$ using a polynomial hash function (backward):

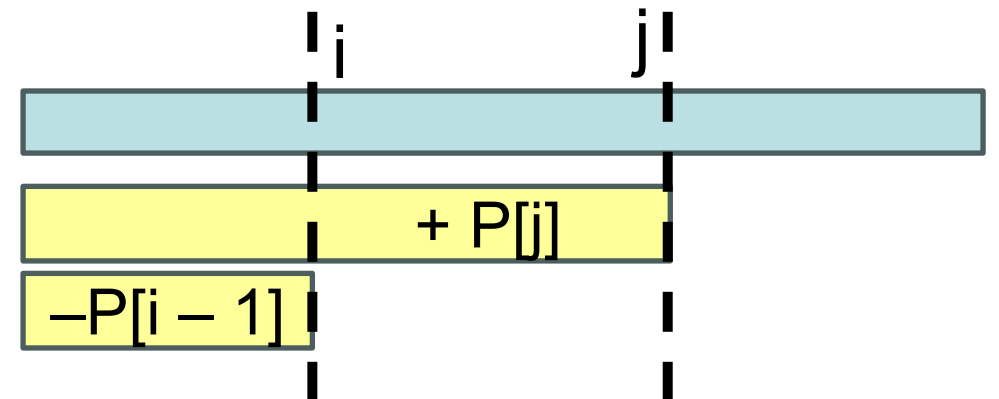
$$(A[0] x^{N-1} + \dots + A[N-3] x^2 + A[N-2] x + A[N-1]) \bmod M$$

- Let's precompute and store “prefix sums”. E.g.,

$$P[j] = (A[0] x^j + \dots + A[j-2] x^2 + A[j-1] x + A[j]) \bmod M$$

- Note: easy to compute $P[j] = (x P[j-1] + A[j]) \bmod M$ in $O(1)$ time.
- Can now find hash of any substring $A[i..j]$ in $O(1)$ time!

$$\text{hash}(A[i..j]) = (P[j] - x^{j-i+1} P[i-1]) \bmod M$$



Dynamic Sequence Hashing

- Hash an array $A[]$ using a polynomial hash function:
$$(A[0] + A[1]x + A[2]x^2 + \dots + A[N-1]x^{N-1}) \bmod M$$
- Easy to update hash in $O(1)$ time if add or remove at ends of array.
 - E.g., if your array represents a stack, queue, or deque
- Similarly easy if you modify an element in the middle.
- Can handle inserts and deletes in middle of sequence in $O(\log n)$ time each by encoding it in a BST, with every subtree augmented with a hash of the sequence it encodes. Also allows splits / joins.